

- The `immediate` mode will abort every PostgreSQL process, including client connections, and shut down the cluster in a dirty way, meaning that the server will need some specific activity on the restart to clean up such dirty data (more on this in the next chapters).

In any case, once a stop command is issued, the server will not accept any new incoming connections from clients, and depending on the stop mode you have selected, existing connections will be terminated. The default stop mode, if none is specified, is `fast`, which forces an immediate disconnection of the clients but ensures data integrity.

If you want to change the stop mode, you can use the `-m` flag, specifying the mode name, as follows:

```
$ pg_ctl stop -m smart
waiting for server to shut down..... done
server stopped
```

In the preceding example, the `pg_ctl` command will wait, printing a dot every second until all the clients disconnect from the server. In the meantime, if you try to connect to the same cluster from another client, you will receive an error, because the server has entered the stopping procedure:

```
$ psql
psql: error: could not connect to server: FATAL: the database system is
shutting down
```

It is possible to specify just the first letter of the stop mode instead of the whole word; so, for instance, `s` for `smart`, `i` for `immediate`, and `f` for `fast`.

PostgreSQL processes

You have already learned how the `postmaster` is the root of all PostgreSQL processes, but as explained in *Chapter 1, Introduction to PostgreSQL*, PostgreSQL will launch multiple different processes at startup. These processes are in charge of keeping the cluster operational and in good health. This section provides a glance at the main processes you can find in a running cluster, allowing you to recognize each of them and their respective purposes.

If you inspect a running cluster from the operating system point of view, you will see a bunch of processes tied to PostgreSQL:

```
$ pstree -p postgres
postgres(1)─postgres(34)
              │
              ├─postgres(35)
              └─postgres(37)
```

```

└─postgres(38)
└─postgres(39)

$ ps -C postgres -af
postgres      1      0  0 11:08 ?        00:00:00 postgres
postgres     34      1  0 11:08 ?        00:00:00 postgres: checkpointer
postgres     35      1  0 11:08 ?        00:00:00 postgres: background
writer
postgres     37      1  0 11:08 ?        00:00:00 postgres: walwriter
postgres     38      1  0 11:08 ?        00:00:00 postgres: autovacuum
launcher
postgres     39      1  0 11:08 ?        00:00:00 postgres: logical
replication launcher

```



The PID numbers reported in these examples refer to the Docker container, where the first PostgreSQL process has a PID equal to 1. On other machines, you will get different PID numbers.

As you can see, the process with PID 1 is one that spawns several other child processes and hence is the first and main PostgreSQL process launched, and as such, is usually called `postmaster`. The other processes are as follows:

- `checkpointer` is the process responsible for executing the checkpoints, which are points in time where the database ensures that all the data is actually stored persistently on the disk.
- `background writer` is responsible for helping to push the data out of the memory to permanent storage.
- `walwriter` is responsible for writing out the **Write-Ahead Logs (WALs)**, the logs that are needed to ensure data reliability even in the case of a database crash.
- `logical replication launcher` is the process responsible for handling logical replication.

Depending on the exact configuration of the cluster, there could be other processes active:

- **Background workers:** These are processes that can be customized by the user to perform background tasks.
- **WAL receiver and/or WAL sender:** These are processes involved in receiving data from or sending data to another cluster in replication scenarios.

Many of the concepts and aims of the preceding process list will become clearer as you progress through the book's chapters, but for now, it is sufficient that you know that PostgreSQL has a few other processes that are always active without any regard to incoming client connections.

When a client connects to your cluster, a new process is spawned: this process, named the back-end process, is responsible for serving the client requests (meaning executing the queries and returning the results). You can see and count connections by inspecting the process list:

```
$ ps -C postgres -af
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
postgres	1	0	0	11:08	?	00:00:00	postgres
postgres	34	1	0	11:08	?	00:00:00	postgres: checkpointer
postgres	35	1	0	11:08	?	00:00:00	postgres: background writer
postgres	37	1	0	11:08	?	00:00:00	postgres: walwriter
postgres	38	1	0	11:08	?	00:00:00	postgres: autovacuum launcher
postgres	39	1	0	11:08	?	00:00:00	postgres: logical replication launcher
postgres	40	1	0	04:35	?	00:00:00	postgres: postgres postgres [local] idle

If you compare the preceding list with the previous one, you will see that there is another process with PID 40: this process is a backend process. In particular, this process represents a client connection to the database named postgres.



PostgreSQL uses a process approach to concurrency instead of a multi-thread approach. There are different reasons for this: most notably, the isolation and portability that a multi-process approach offers. Moreover, on modern hardware and software, forking a process is no longer so much of an expensive operation.

Therefore, once PostgreSQL is running, there is a tree of processes that roots at postmaster. The aim of the latter is to spawn new processes when there is the need to handle new database connections, as well as to monitor all maintenance processes to ensure that the cluster is running fine.

Connecting to the cluster

Once PostgreSQL is running, it awaits incoming database connections to serve; as soon as a connection comes in, PostgreSQL serves it by connecting the client to the right database. This means that to interact with the cluster, you need to connect to it. However, you don't connect to the whole cluster; rather, you ask PostgreSQL to interact with one of the databases the cluster is serving. Therefore, when you connect to the cluster, you need to connect to a specific database. This also means that the cluster must have at least one database from the very beginning of its life.

When you initialize the cluster with the `initdb` command, PostgreSQL builds the filesystem layout of the `PGDATA` directory and builds two template databases, named `template0` and `template1`. The template databases are used as a starting point to clone other new databases, which can then be used by normal users to connect to. In a freshly installed PostgreSQL cluster, you usually end up with a `postgres` database, used to allow the database administrator user `postgres` to connect to and interact with the cluster.

To connect to one of the databases, either a template or a user-defined one, you need a *client* to connect with. PostgreSQL ships with `psql`, a command-line client that allows you to connect, interact with, and administer databases and the cluster itself.

Other clients do exist, but they will not be discussed in this book. You are free to choose the client you like the most, since every command, query, and example shown in the book will run with no exception under every compatible client.

While connecting interactively to the cluster is an important task for a database administrator, often, developers need their own applications to connect to the cluster. To achieve this, the applications need a so-called *connection string*, a URI indicating all the required parameters to connect to the database.

This section will explain all the preceding concepts, starting from the template databases and then showing the basic usage of `psql` and the connection string.

The template databases

The `template1` database is the first database created when the system is initialized, and then it is cloned into `template0`. This means that the two databases are, at least initially, identical, and the aim of `template0` is to act as a safe copy for rebuilding in case it is accidentally damaged or removed.

You can inspect available databases using the `psql -l` command. On a freshly installed installation, you will get the following three databases:

```
$ psql -l                               List of databases
  Name      | Owner   | Encoding | Collate  | Ctype    | ICU Locale |
  Locale Provider | Access privileges
-----+-----+-----+-----+-----+-----+-----
 postgres   | postgres | UTF8     | it_IT.UTF-8 | it_IT.UTF-8 |             |
 libc       |          |          |             |             |             |
 template0  | postgres | UTF8     | it_IT.UTF-8 | it_IT.UTF-8 |             |
 | libc     |          | =c/postgres |             |             |             |
 |          |          |          |             |             |             |
 | postgres=CtC/postgres
 template1  | postgres | UTF8     | it_IT.UTF-8 | it_IT.UTF-8 |             |
 | libc     |          | =c/postgres |             |             |             |
 |          |          |          |             |             |             |
 | postgres=CtC/postgres
(3 rows)
```



In the Docker image, you will also see the `forumdb` database, which has been automatically created for you to let you interact with other examples.

It is interesting to note that, alongside the two template databases, there's a third database that is created during the installation process: the `postgres` database. That database belongs to the `postgres` user, which is, by default, the only database administrator created during the initialization process. This database is a *common space* to be used for connections instead of the template databases.

The name **template** indicates the real aim of these two databases: when you create a new database, PostgreSQL clones a template database as a **common base**. This is somewhat like creating a user home directory on Unix systems: the system clones a **skeleton** directory and assigns the new copy to the user. PostgreSQL does the same—it clones `template1` and assigns the newly created database to the user that requested it.

What this also means is that whatever object you put into `template1`, you will find the very same object in freshly created databases. This can be really useful for providing a common **base database** and having all other databases brought to life with the same set of attributes and objects.

Nevertheless, you are not forced to use `template1` as the base template; in fact, you can create your own databases and use them as templates for other databases. However, please keep in mind that, by default, (and most notably on a newly initialized system), the `template1` database is the one that is cloned for the first databases you will create.

Another difference between `template1` and `template0`, apart from the former being the default for new databases, is that you cannot connect to the latter. This is in order to prevent accidental damage to `template0` (the safety copy).

It is important to note that the cluster (and all user-defined databases) can work even without the template databases—the `template1` and `template0` databases are not fundamental for the other databases to run. However, if you lose the templates, you will be required to use another database as a template every time you perform an action that requires it, such as creating a new database.



Template databases are not meant for interactive connections, and you should not connect to the template databases unless you need to customize them. PostgreSQL will present as a *skeleton* for another database if there are active connections to it.

The psql command-line client

The `psql` command is the command-line interface that ships with every installation of PostgreSQL. While you can certainly use a graphical user interface to connect and interact with the databases, a basic knowledge of `psql` is mandatory in order to administer a PostgreSQL cluster. In fact, a specific `psql` version is shipped with every release of PostgreSQL; therefore, it is the most up-to-date client speaking the same language (i.e., protocol) of the cluster. Moreover, the client is lightweight and useful even in emergency situations when a GUI is not available.

`psql` accepts several options to connect to a database, mainly the following:

- `-d`: The database name
- `-U`: The username
- `-h`: The host (either an IPv4 or IPv6 address or a hostname)

If no option is specified, `psql` assumes your operating system user is trying to connect to a database with the same name, and a database user with a name that matches the operating system on a local connection. Take the following connection:

```
$ id
uid=999(postgres) gid=999(postgres) groups=999(postgres),101(ssl-cert)

$ psql
psql (16.0)
Type "help" for help.

postgres=#
```

This means that the current operating system user (`postgres`) has required `psql` to connect to a database named `postgres` via the PostgreSQL user named `postgres` on the local machine. Explicitly, the connection could have been requested as follows:

```
$ psql -U postgres -d postgres
psql (16.0)
Type "help" for help.

postgres=#
```

The first thing to note is that once a connection has been established, the command prompt changes: `psql` reports the database to which the user has been connected (`postgres`) and a sign to indicate they are a superuser (`#`). In the case that the user is not a database administrator, a `>` sign is placed at the end of the prompt.

If you need to connect to a database that is named differently by your operating system username, you need to specify it:

```
$ psql -d template1
psql (16.0)
Type "help" for help.

template1=#
```

Similarly, if you need to connect to a database that does not correspond to your operating username with a PostgreSQL user that is different from your operating system username, you have to explicitly pass both parameters to `psql`:

```
$ id
uid=999(postgres) gid=999(postgres) groups=999(postgres),101(ssl-cert)

$ psql -d template1 -U luca
psql (16.0)
Type "help" for help.

template1=>
```

As you can see from the preceding example, the operating system user `postgres` has connected to the `template1` database with the PostgreSQL user `luca`. Since the latter is not a system administrator, the command prompt ends with the `>` sign.

To quit from `psql` and close the connection to the database, you have to type `\q` or `quit` and press **Enter** (you can also press **CTRL + D** to exit on any Unix and Linux machines):

```
$ psql -d template1 -U luca
psql (16.0)
Type "help" for help.

template1=> \q
$
```

Entering SQL statements via `psql`

Once you are connected to a database via `psql`, you can issue any statement you like. Statements must be terminated by a semicolon, indicating that the next **Enter** key will execute the statement. The following is an example where the **Enter** key has been emphasized:

```
$ psql -d template1 -U luca
psql (16.0)
Type "help" for help.
template1=> SELECT current_time; <ENTER>
current_time
```

```
-----
06:04:57.435155-05
```

```
(1 row)
```



SQL is a case-insensitive language, so you can enter statements in either uppercase, lowercase, or a mix. The same rule applies to column names, which are case-insensitive. If you need to have identifiers with specific cases, you need to quote them in double quotes.

Another way to execute the statement is to issue a `\g` command, again followed by `<ENTER>`. This is useful when connecting via a terminal emulator that has keys remapped:

```
template1=> SELECT current_time \g <ENTER>
```

```
current_time
```

```
-----
06:07:03.328744-05
```

```
(1 row)
```

Until you end a statement with a semicolon or `\g`, `psql` will keep the content you are typing in the *query buffer*, so you can also edit multiple lines of text as follows:

```
template1=> SELECT
```

```
template1-> current_time
```

```
template1-> ;
```

```
current_time
```

```
-----
06:07:28.908215-05
```

```
(1 row)
```

Note how the `psql` command prompt has changed on the lines following the first one: the difference is there to remind you that you are editing a multi-line statement and `psql` has not (yet) found a statement terminator (either a semicolon or the `\g`).

One useful feature of the `psql` query buffer is the capability to edit the content of the query buffer in an external editor. If you issue the `\e` command, your favorite editor will pop up with the content of the last-edited query. You can then edit and refine your SQL statement as much as you want, and once you exit the editor, `psql` will read what you have produced and execute it. The editor to use is chosen with the `EDITOR` operating system environment variable.

It is also possible to execute all the statements included in a file or edit a file before executing it. As an example, assume the `test.sql` file has the following content:

```
$ cat test.sql

SELECT current_database();
SELECT current_time;
SELECT current_role;
```

The file has three very simple SQL statements. In order to execute the whole file at once, you can use the `\i` special command followed by the name of the file:

```
template1=> \i test.sql
current_database
-----
template1

(1 row)

      current_time
-----
06:08:43.077305-05

(1 row)

      current_role
-----
luca

(1 row)
```

As you can see, the client has executed, one after the other, every statement within the file. If you need to edit the file without leaving `psql`, you can issue `\e test.sql` to open your favorite editor, make changes, and come back to the `psql` connection.